Filière :	**Développement des Systèmes d'Information - DSI -**	**Durée :**	**4 heures**
Épreuve :	**Développement des applications informatiques**	**Coefficient :**	**45**

CONSIGNES

- **Le sujet comporte 4 dossiers ;**
- **Chaque dossier doit être traité dans une feuille séparée.**

BARÈME DE NOTATION

Dossier 1 : Gestion des départements	14 points
Dossier 2 : Gestion des enseignants	8 points
Dossier 3 : Gestion des demandes de formations	10 points
Dossier 4 : Gestion des stages	8 points
Total	**40 points**

- **Il sera pris en considération la qualité de la rédaction lors de la correction.**
- **Aucun document n'est autorisé.**

ÉTUDE DE CAS : SOCIÉTÉ DE FORMATION ET RECHERCHE AGRONOMIQUE

La société « **Sais Agro** » est une société de formation et recherche agronomique. Elle possède plusieurs centres de recherches. Elle assure la formation de jeunes agriculteurs dont certains continueront dans l'un des instituts ou centres de recherche pour devenir ingénieur agronome ou chercheur.

Les membres de cette société communiquent et échangent des données entre eux par le biais d'une plateforme Internet.

Cette étude de cas est composée de 4 dossiers dont le détail est le suivant :

- **Dossier 1** : Gestion des départements ;
- **Dossier 2** : Gestion des enseignants ;
- **Dossier 3** : Gestion des demandes de formations ;
- **Dossier 4** : Gestion des stages.

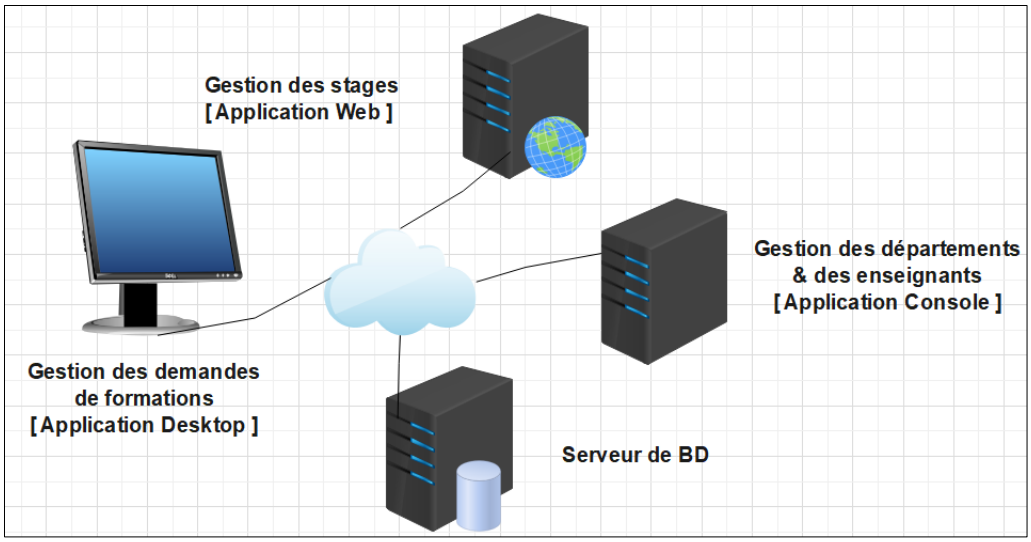


***Figure 1**: Espaces de la plateforme Internet*

DOSSIER I : GESTION DES DÉPARTEMENTS *(14 points)*

La société **« Sais Agro »** souhaite modéliser ses étudiants ainsi que la répartition de ses enseignants au sein des différents départements. Pour ce faire, on adopte le diagramme de classes de la figure 1 respectant les règles suivantes :

- Une personne peut être soit un étudiant soit un enseignant ;
- Un département contient un ou plusieurs enseignants.

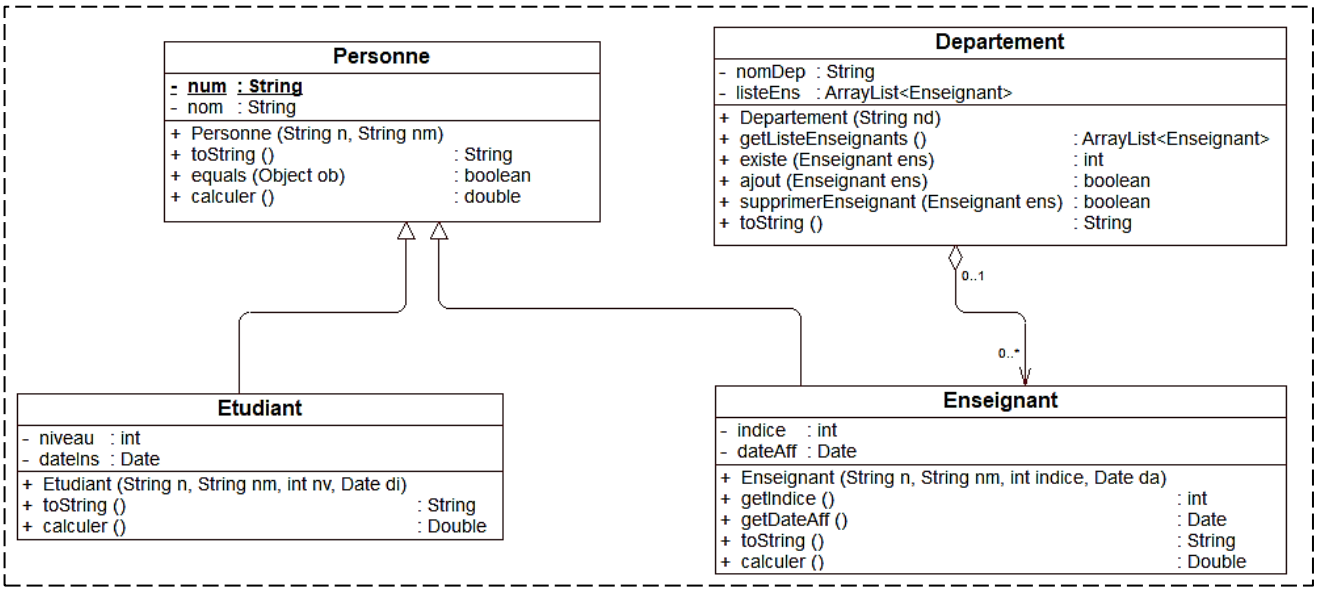


***Figure 2** : Diagramme de classes*

1. Le détail de la classe **« Personne »** est le suivant :

```
public abstract class Personne implements Serializable {
        private String num;
        private String nom;
        public Personne(String num , String nom) { ... }
        @Override
        public String toString() { ... }
        @Override
        public boolean equals(Object Ob) { ... }
        public abstract double calculer();
}
```

a. Définir le constructeur avec arguments. *(0,5 pt)*

b. Donner la définition de la méthode **« toString »**, qui retourne une chaîne de caractères porteuse d'informations sous la forme suivante : *(0,5 pt)*

```
Numéro : xxx, Nom : xxx
```

c. Redéfinir la méthode **« equals »** qui permet de comparer deux personnes selon l'attribut **« num »**. (*La comparaison est insensible à la casse*). *(0,5 pt)*

2. Le détail de la classe **« Enseignant »** est le suivant :

```
public class Enseignant extends Personne{
   private int indice;
   private Date dateAff;
   public Enseignant(String num , String nom , int ind , Date dtAff) { ... }
   public int getIndice()  {  ...  }
   public Date getDateAff() { ... }
       @Override
   public String toString()  {  ... }
      @Override
   public double calculer()  {    ... }
}
```

a. Définir le constructeur avec arguments qui génère une exception **« ErreurIndice »** *(à définir)* si **l'indice** est **inférieur** à **200** ou **supérieur** à **1000**. *(1 pt)*

b. Redéfinir la méthode **« toString »**, qui retourne une chaîne de caractères porteuse d'informations sous la forme suivante : *(1 pt)*

```
Numéro : xxx, Nom : xxx, Indice : xxx, Date d'affectation : jj/mm/aaaa
```

c. Définir la méthode **« calculer »** qui permet de calculer et de retourner le **salaire net** de l'enseignant. Le salaire net est calculé de la façon suivante : *(1,5 pt)*

```
salaireNet = salaireBrute-IGR
Tel que :
      salaireBrute = indice*25
      IGR = salaireBrute*38%
```

3. Le détail de la classe **« Etudiant »** est le suivant :

```
public class Etudiant extends Personne{
      private int niveau;
      private Date dateIns;
      public Etudiant(String num, String nom, int niv, Date ins) { ... }
      @Override
      public String toString() { ... }
      @Override
      public double calculer()  { ... }
}
```

a. Définir le constructeur avec arguments. Ce constructeur génère une exception **« ErreurNiveau »** si le **niveau** est différent de 1, 2 ou 3. *(1 pt)*
(*On suppose que cette classe d'exception existe déjà et ayant un seul constructeur avec un argument de type String*)

b. Redéfinir la méthode **« toString »**, qui retourne une chaîne de caractères porteuse d'informations sous la forme suivante : *(0,5 pt)*

```
Numéro : xxx, Nom : xxx, Niveau : xxx, Date d'inscription : jj/mm/aaaa
```

c. Définir la méthode **« calculer »** qui calcule et retourne la **bourse** de l'étudiant. La bourse est calculée de la façon suivante : **Bourse = 1000+(150*niveau)** *(1 pt)*

4. Le détail de la classe **« Departement »** est le suivant :

```
public class Departement {
      private String nomDep;
      private ArrayList<Enseignant> listeEns;

      public Departement(String nomDep)  { ... }

      public ArrayList<Enseignant> getListeEnseignants() { ... } //Code non demandé

      public int existe (Enseignant ens) { ... }

      public boolean ajout(Enseignant ens) { ... }

      public boolean supprimerEnseignant(Enseignant ens)  { ... }
    @Override
      public String toString()  { ... }
}
```

Donner le code des méthodes suivantes :

a. Un constructeur avec un seul paramètre **(nomDep)** qui permet de mémoriser le nom du département et d'initialiser la liste des enseignants. *(1 pt)*

b. **« existe »** : qui retourne **l'indice dans ArrayList « listeEns »** de l'enseignant reçu comme argument. S'il n'existe pas elle retourne « **-1** ». *(1 pt)*

c. **« ajout»** : qui ajoute un enseignant à la liste des enseignants et retourne l'état de cette opération.
NB : avant l'ajout, il faut vérifier que l'enseignant n'existe pas dans la liste. *(1,5 pt)*

d. **« supprimerEnseignant »** : permet de supprimer un enseignant passé en paramètre, et retourne « **true** » si l'enseignant est bien supprimé et « **false** » dans le cas contraire. *(1 pt)*

e. Redéfinir la méthode **« toString »**, qui retourne une chaîne porteuse d'informations sous la forme : *(2 pts)*

```
Nom Département :  xxx
Liste des enseignants :
 Numéro : xxx, Nom : xxx, Indice : xxx, Date d'affectation : jj/mm/aaaa
 ...
```

DOSSIER II : GESTION DES ENSEIGNANTS *(8 points)*

La gestion des enseignants est assurée grâce à une application client-serveur dont la structure est donnée ci-après :

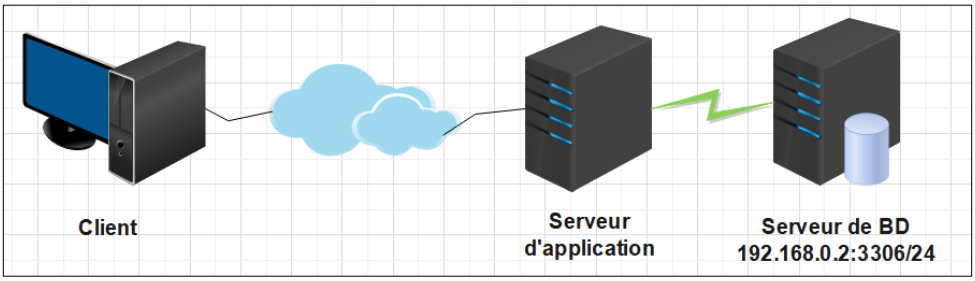


Figure 3 *: Architecture Client-Serveur*

➢ **SERVEUR D'APPLICATION**

Dans un serveur d'application, on veut créer une classe permettant la connexion à la base de données **« Academie »** en utilisant l'API **JDBC**. La structure de cette classe est la suivante :

Connexion
- cn : Connection - URL : String - User : String - Password : String

Figure 4 *: Classe* ***« Connexion »***

La base de données **« Academie »** se trouve dans un serveur ayant l'adresse **IP** : **192.168.0.2** et le port d'accès : **3306**. On se limite à la gestion des enseignants. Voici la structure de la table **« Enseignant »** :

	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
☐	1	**num** 🔑	int(11)			Non	*Aucun(e)*		AUTO_INCREMENT
☐	2	**nom**	varchar(25)	utf8mb4_general_ci		Non	*Aucun(e)*		
☐	3	**indice**	int(11)			Non	*Aucun(e)*		
☐	4	**dateAff**	date			Non	*Aucun(e)*		

Figure 5 *: Structure de la table* ***Enseignant***

Le détail de la classe connexion est le suivant :

```
import java.sql.*;
public class Connexion {
      private String URL,User, Password ;
      private Connection cn;
      public Connexion(String URL, String User, String Password) {
           this.URL=URL;
          this.User=User;
          this.Password=Password;
      }

      public boolean connecter()
                {   ……————————————   }

      public ResultSet lireEnseigants(int indice)
                {   ……————————————   }

      public boolean ajoutEnseignant(Enseignant ens)
                {   ……————————————   }

      public boolean fermer()
                {   ……————————————   }
}
```

Définir les méthodes suivantes de la classe **« Connexion »** :

1. **« connecter »** : pour se connecter à la base de données en se basant sur les attributs déjà initialisés par le constructeur. Cette méthode retourne **« true »** en cas de succès ou **« false »** en cas d'échec. *(1,5 pt)*
2. **« lireEnseigants» :** permettant de retourner, sous-forme d'un objet **ResultSet**, tous les enseignants ayant l'indice reçu comme argument. En cas d'échec, elle retournera **« null ».** *(1,5 pt)*
3. **« ajoutEnseignant» :** permettant d'ajouter un nouvel enseignant. Cette méthode retourne **« true »** en cas de succès ou **« false »** en cas d'échec. *(1,5 pt)*
4. **« fermer »** qui permet de fermer la connexion de base de données. Cette méthode retourne **« true »** en cas de succès ou **« false »** en cas d'échec. *(0,5 pt)*

➢ APPLICATION CLIENT

Le centre de formation a mis en place une application **Client/serveur** pour la consultation de la liste des enseignants en utilisant les sockets en mode connecté. Une partie de cette application permet au responsable de consulter la liste des enseignants de son département à partir de son poste en se connectant au serveur d'application. Il saisit un **« indice »** afin de recevoir la liste des enseignants ayant cet indice.

La classe **« Client »** se présente comme suit :

Client
- IP : String - Port : int - s : Socket - out : PrintWriter - in : ObjectInputStream
+ Client(…) + setconnexion() : boolean + demandeListeEnseignants(…) : ArrayList<Enseignant>

Figure 6** : Classe **« Client »

```
public class Client {
      private String IP;
      private int Port;
      private Socket s=null;
      private PrintWriter out=null;
      private ObjectInputStream in=null;

      public Client(String IP, int Port) {
             this.IP=IP;
             this.Port=Port;
       }
      public  boolean setConnexion()   { ... }
      public ArrayList<Enseignant> demandeListeEnseignants(int indice) { ... }
      public static void main(String[] args){ ...  } // code non demandé
}
```

5. Écrire la méthode **« setConnexion »** qui permet d'établir la connexion **TCP/IP** avec le serveur d'application et d'initialiser les objets **« out »** et **« in »**. La méthode retourne **« true »** en cas de succès et **« false »** dans le cas échéant. *(1,5 pt)*

6. Écrire la méthode **« demandeListeEnseignants »** qui envoie au serveur un indice. La réponse du serveur est une collection **« ArrayList »** d'enseignants qui est la valeur de retour de cette méthode. *(1,5 pt)*

DOSSIER III : GESTION DES DEMANDES DE FORMATIONS *(10 points)*

Le responsable des formations, de la société **« Sais Agro »**, souhaite disposer d'une application informatique, sous Visual Basic, permettant le suivi des demandes de formation. Les demandeurs (*étudiants, stagiaires …*) peuvent choisir le cours et attendre l'accord ou le refus du responsable de la formation.

Le responsable, suite à des critères bien déterminés, accepte ou refuse la demande du candidat.

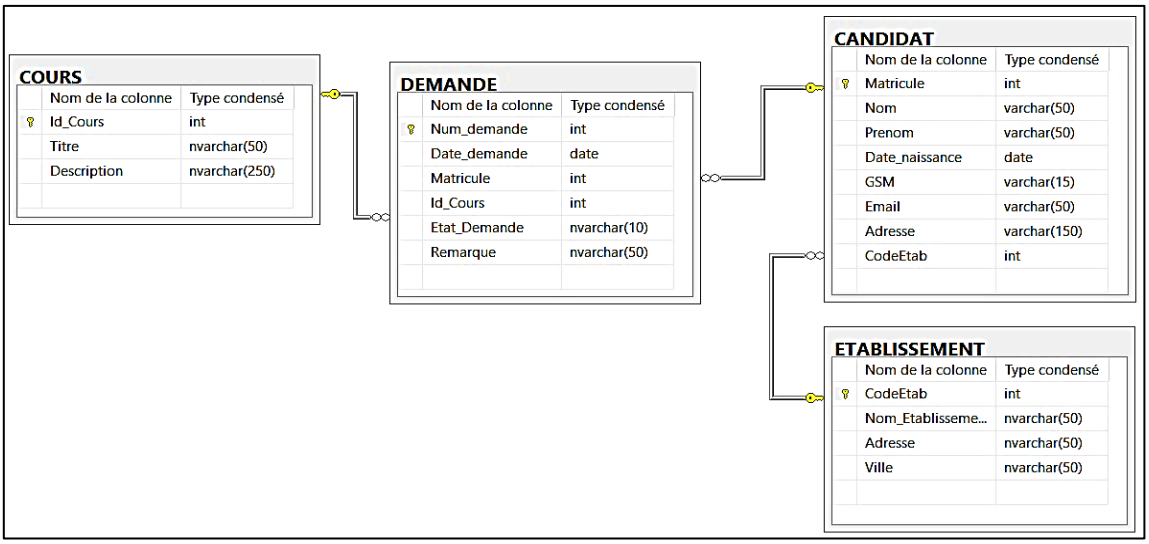


***Figure 7** : Partie du MLDR de la base de données « BD_Formations» implantée dans Microsoft SQL Server*

Remarque : Dans la table « **DEMANDE** », la propriété **« Etat_Demande »** permet de connaître l'état de traitement de la demande. Cette propriété peut prendre l'une des valeurs suivantes : ***« en cours »*** (*valeur par défaut*), ***« acceptée »***, ***« en attente »*** et ***« refusée »***.

Consigne : Veuillez indiquer le mode de connexion utilisé (***mode connecté*** *ou* ***non connecté***).
Toutes les questions doivent être traitées avec le mode de connexion choisi.

1. Dans un module,

a) Déclarer les objets de connexion avec la base de données. *(0,5 pt)*

b) Donner le code de la procédure « **Connexion** » qui établit une connexion avec la base de données (*la gestion des exceptions est obligatoire*). *(1,5 pt)*

NB : *Nom du serveur : Srv_Format*, l'authentification est celle du Windows.

```
Public Module Module1
    ...
   Public sub Connexion ( )
           ...
   End sub
End Module
```

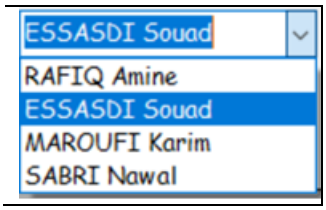
2. Dans l'IHM d'enregistrement de nouvelles demandes, créer les méthodes suivantes :

a) Écrire le code de la fonction **« Generer_NumDem() »** qui permet de retourner le numéro pour une nouvelle demande. Ce numéro est la plus grande valeur du champ **« Num_demande »** de la table **« DEMANDE »** incrémentée par **1**. *(1,5 pt)*

```
Private Function Generer_NumDem() ...
     ...
End Function
```

b) Écrire le code de la procédure **« Remplir_Candidats() »** permettant de remplir le **ComboBox** nommé **« CmbCandidat »** par les noms complets de tous les candidats. *(1,5 pt)*

```
Private Sub Remplir_Candidats()
 ...
End Sub
```



c) Écrire le code de la procédure **«Enregistrer_Demande»**, qui permet d'ajouter une nouvelle demande (*On suppose que les valeurs des clés étrangères sont valides et que la date de la nouvelle demande est celle du système*). *(2 pts)*

```
Private Sub Enregistrer_Demande(ByVal mat As Integer, ByVal  idCours As Integer, ByVal
remarque As String)
...
End Sub
```

3. Dans l'IHM de validation des demandes, créer les méthodes suivantes :

   a. Écrire le code de la procédure **« AfficherDemandes»** qui liste, dans l'objet **DataGridView** nommé **« DGVListe»**, toutes les demandes ayant l'état : **« *en cours* »** ou **« *en attente* »** et dont le titre du cours commence par la chaine du paramètre reçu. *(1,5 pt)*

```
Private Sub AfficherDemandes(ByVal titreCours As String)
...
End Sub
```

	Numéro	Date Demande	Nom Complet	Titre	État Demande
▶					

   b. Écrire le code de la procédure **« Modif_EtatDemande»** qui permet de modifier l'état d'une demande donnée. La procédure prend en argument le numéro de la demande **« numDem»** et son nouvel état **« newEtat»**. *(1,5 pt)*

```
Private Sub  Modif_EtatDemande(ByVal numDem As Integer, ByVal newEtat As String)
...
End Sub
```

DOSSIER IV : GESTION DES STAGES *(8 points)*

Dans le cadre d'informatisation des services, la société **« Sais Agro »** souhaite mettre en place une application web pour gérer les stages. Dans ce contexte, une base de données nommée **«Gestion_stages»** est implémentée sous MySQL dont une partie de son MLDR est le suivant :

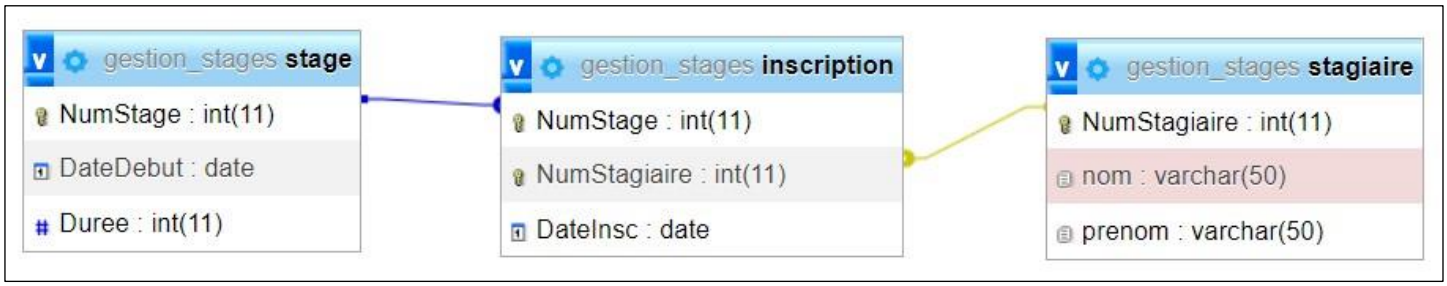


***Figure 8** : Partie du MLDR de la base de données « Gestion_stages» implantée dans MySQL*

Informations complémentaires :

- Le champ « **NumStagiaire** » de la Table « **STAGIAIRE** » est **AUTO_INCREMENT**.
- Le champ « **NumStage** » de la Table « **STAGE** » est **AUTO_INCREMENT**.

1. Dans un fichier nommé **« connexion.php »**, donner le code PHP pour se connecter à la base de données **« Gestion_stages »**, sachant que : *(1,5 pt)*
   - Nom du serveur : **« serveurHost »**
   - Nom utilisateur : **« user1 »**
   - Mot de passe : **« user1@1234 »**.

2. La page d'accueil **« index.html »,** qui permet d'accéder aux différents espaces du site de gestion des stages, est la suivante :

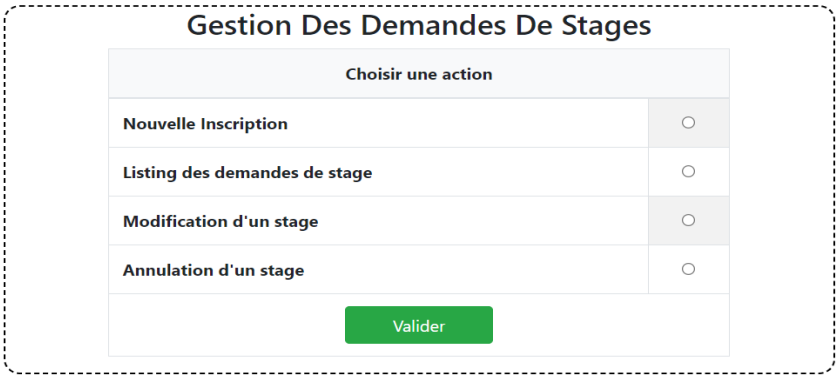


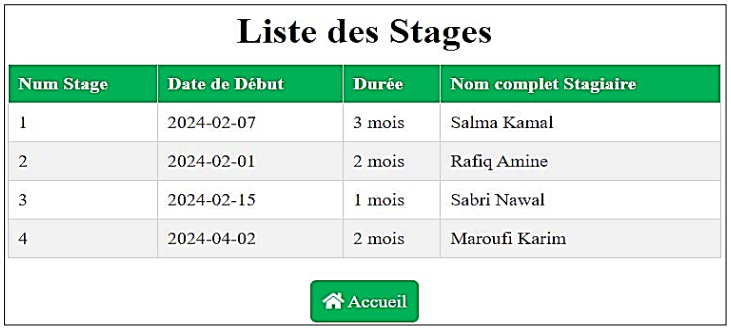
Figure 9 : *Page « index.html »*

N.B : Le fichier de traitement du formulaire de cette page est nommé **« traitement.php »**. La méthode de transfert des données est **« POST »**. Le nom des boutons radio est **« choix »**. Les valeurs (**value**) des boutons radio sont : **« a »**, **« b »**, **« c »** et **« d »**.

Créer le fichier **« traitement.php »** qui reçoit les informations du formulaire de la page **« index.html »**. Ce fichier assure les redirections selon le choix de l'action à réaliser : *(1,5 pt)*

- ✓ Nouvelle inscription ➔ **« inscription.php »**
- ✓ Listing des demandes de stage ➔ **« listing.php »**
- ✓ Modification d'un stage ➔ **« modification.php »**
- ✓ Annulation d'un stage ➔ **« annulation.php »**

3. Si on choisit l'action **« Listing de demandes de stage »**, une page nommée **« listing.php »** est affichée comme suit :



Liste des Stages

Num Stage	Date de Début	Durée	Nom complet Stagiaire
1	2024-02-07	3 mois	Salma Kamal
2	2024-02-01	2 mois	Rafiq Amine
3	2024-02-15	1 mois	Sabri Nawal
4	2024-04-02	2 mois	Maroufi Karim

Accueil

Figure 10 : *Page « listing.php »*

Le code de cette page est le suivant :

```
<html>
<head> </head>
<body> <h1> Liste des stages </h1>
        <table>
            <thead> <tr> <th> Num Stage </th> <th> Date de Début </th>
                <th> Durée </th><th> Nom complet Stagiaire </th></tr>  </thead>
            <tbody>
                    <?php
                        include "connexion.php";
                        include "lister.php";    ?>
            </tbody> </table>
                <a href="index.html">Accueil</a>
</body>
</html>
```

Donner le code du fichier **« Lister.php »** permettant de lister tous les stages. *(2 pts)*

4. Si on choisit l'action **« Nouvelle inscription »,** une page apparait nommée **« inscription.php »** contient le formulaire d'inscription suivant :

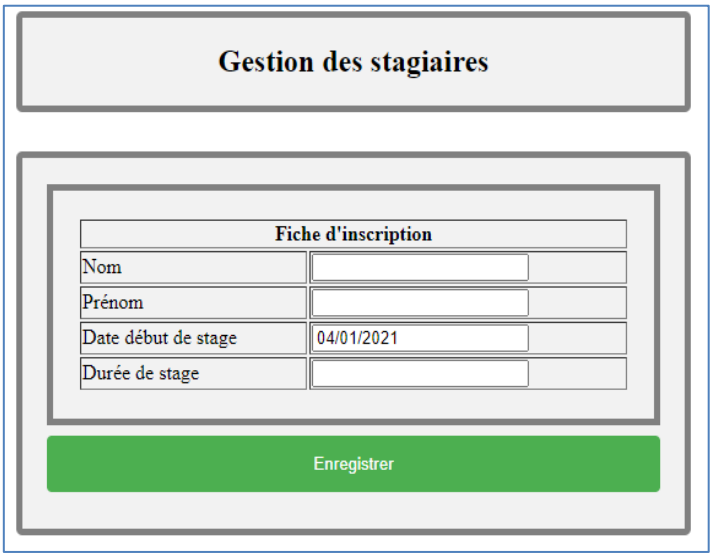


Figure 11 *: Page « inscription.php »*

N.B : Le fichier de traitement de formulaire de cette page est nommé **« trInscription.php »**. La méthode de transfert des données est **«POST »**. Les noms des champs de texte sont : **« nom »**, **« prenom »**, **« dateDebut »** et **« duree »**.

Donner le code de fichier **« trInscription.php »** pour sauvegarder les informations saisies dans le formulaire dans les **trois** tables de la base de données **« Gestion_stages »**. On rappelle que la date d'inscription est celle de la date système du serveur Web. *(3 pts)*

Rappel :

- La fonction **« date("Y-m-d") »** : retourne la date système.
- La fonction permettant de récupérer le dernier identifiant généré par le SGBDR est :
  - En procédural : **mysqli_insert_id ($con)** ;
  - En orienté objet ***mysqli***: **$con→insert_id**;
  - En orienté objet ***PDO*** : **$con→lastinsertid()**.

% Bon courage %